

Changelog

v1.10.0 — 2026-06-29 — Auto-applied per-alias session color + coverage/wiring

Added

Fixed

Verified

v1.9.2 — 2026-06-29 — Hermetic CLAUDE_BIN resolver test

Fixed

Verified

v1.9.1 — 2026-06-29 — CLAUDE_BIN resolves across per-host install locations

Fixed

Verified

v1.9.0 — 2026-06-29 — Per-project auto-sessions that actually work live + zero-coverage tests

Fixed

Added

Verified

v1.8.1 — 2026-06-29 — Merge-engine correctness + portability hardening

Fixed

Added

Verified

v1.8.0 — 2026-06-29 — Alias isolation + token-limit guard + per-project auto-sessions

Fixed

Added

Verified

v1.7.12 — 2026-06-28 — One-line curl installer

Added

Verified

v1.7.11 — 2026-06-28 — Round-4: coverage-gap regression tests, toon recursion guard, arg validation

Fixed

Added — coverage-gap regression tests (the same class that let enable-plugins

Verified

v1.7.10 — 2026-06-28 — Round-3 audit: enable-plugins bug fix, path-traversal guards, proxy robustness

Fixed

Security (defense-in-depth)

Robustness

Verified

Audit (round 3) — verified clean

v1.7.9 — 2026-06-28 — Hardening round 2: injection-safe alias writes, broadened secret redaction, docs accuracy, shellcheck 0

Security

Fixed

Docs
Quality
Verified
v1.7.8 — 2026-06-28 — Secret hygiene (argv + committed-proof leaks), dead-code fix, coverage tests
Security
Fixed
Added
Verified
v1.7.7 — 2026-06-28 — Portable realpath (BSD portability hardening), set -u edge fix, regression tests
Fixed
Added
Verified
Audit findings (v1.7.6 — no code change required)
v1.7.6 — 2026-06-28 — Always-non-interactive execution, alias-file integrity, macOS/bash-3.2 portability, 4-host rollout
Fixed
Added
Multi-host rollout (nezha · mistborn.local · thinker.local · amber.local)
Verified
v1.7.5 — 2026-06-28 — Cross-provider /resume session visibility fix
Fixed
Root Cause Analysis
Verified
Tests
v1.7.4 — 2026-06-26 — Kimi provider fix + AWS IaC MCP disabled by default
Fixed
Changed
Tests
v1.6.6 — 2026-06-21 — TOON integration for token-efficient prompts
Added
Token Savings
Note
Tests
v1.6.5 — 2026-06-21 — Poe proxy fix (alias file + install)
Fixed
Verified
Tests
v1.6.4 — 2026-06-21 — Poe proxy fix for tool compatibility
Fixed
Verified
Tests
v1.6.3 — 2026-06-21 — Poe provider (382 models, 3 aliases)
Added
Verified

v1.6.2 — 2026-06-21 — Chutes provider documentation + model update
 Changed
 Verified

v1.6.1 — 2026-06-21 — cache_control fix + E2E tests
 Fixed
 Added
 Verified working (all aliases tested with "Do you see our codebase?")

v1.6.0 — 2026-06-21 — Multi-alias provider system
 Added
 Changed
 Full test suite
 Usage

v1.5.1 — 2026-06-20 — Linux stat fix + nezha deployment
 Fixed
 Deployment
 Full test suite

v1.5.0 — 2026-06-20 — Cross-alias session visibility
 Added
 Changed
 Full test suite
 How it works
 Performance

v1.4.0 — 2026-06-20 — OpenCode Zen provider alias
 Added
 Changed
 Full test suite
 Honest notes

v1.3.0 — 2026-06-19 — Xiaomi MiMo provider alias
 Added
 Changed
 Full test suite
 Honest notes

v1.2.0 — 2026-06-19 — Z.AI Coding Plan provider alias
 Added
 Changed
 Full test suite

v1.1.0 — 2026-06-16 — Distributed infrastructure + provider verification
 Added
 Changed
 Fixed (LLMsVerifier — shipped as PR #2, applied to deployed builds)
 Verified live (real "Do you see my code?" against production APIs)
 Safety

v1.0.0 — 2026-06-16 — Dynamic provider-alias generator

v1.6.7 — 2026-06-21 — Poe proxy fix for all aliases
 Fixed
 Verified
 Tests

v1.6.8 — 2026-06-21 — Poe proxy gzip fix
Fixed
Verified
Tests
v1.6.9 — 2026-06-21 — Poe proxy \$ref fix for Grok-4
Fixed
Verified
Tests
v1.7.0 — 2026-06-22 — Poe proxy complete fix (all aliases verified)
Fixed
Verified (all three aliases through full Claude Code flow)
Root Cause Analysis
Tests
v1.7.1 — 2026-06-22 — Full validation + release
Fixed
Tests
Release
v1.7.2 — 2026-06-22 — Claude alias verification, full release
Added
Tests

Changelog

All notable changes to the Claude multi-account toolkit.

v1.10.0 — 2026-06-29 — Auto-applied per-alias session color + coverage/wiring

The per-alias session color is now **auto-applied** (it was only a hint in v1.9.x), plus self-healing for a stale CLAUDE_BIN and several closed test-coverage gaps.

Added

- **Auto-applied per-alias session color.** Each bare alias launch now writes the alias's color into the session as an agent-color record — the exact record Claude Code's /color writes — via the new claude-session apply-color, called by cma_run/cma_run_provider (before launch to colour a resumed session; after exit to colour a freshly-created one). Deterministic $\text{md5}(\text{label}) \bmod 8$ over red/blue/green/yellow/purple/orange/pink/cyan: each alias gets a stable, distinct colour, and switching the same session between aliases re-colours it. Verified **LIVE** on claude 2.1.195 — written, idempotent,

persists across `--resume`. (Prompt-bar rendering must be confirmed visually: `/color` is TUI-only and `claude -p '/color x'` is a no-op, so record injection is the only non-interactive mechanism. See [docs/SESSION_COLOR.md](#).)

- Test coverage: `test_install.sh` (executes `install.sh` in a sandbox — symlinks, alias file, idempotency), `test_verify_scripts.sh` (`model_verify.py`
 - `providers-verify.sh`), `test_session` `apply-color` tests (incl. the `set -e` regression), `test_coverage` B7 (`CLAUDE_BIN` resolver), B8 (`CLAUDE_BIN` migration), B9 (`apply-color` wired into both wrappers). `run-proof.sh` now also runs the previously-orphaned `verify_aliases_live.sh`.

Fixed

- **Stale `CLAUDE_BIN` self-heals.** Existing installs whose alias file pointed `CLAUDE_BIN` at a non-existent path (e.g. `~/.local/bin/claude` where `npm` put `claude` in `~/.npm-global/bin` — the `amber.local` case) now rewrite it to a resolved, executable `claude` on the next `install/ensure`.
- A `set -e/pipefail` bug where `apply-color` aborted before writing on a session that had no existing `agent-color` record.

Verified

- Suite **16/16 ALL GREEN; shellcheck 0**. Color injection proven **LIVE** on real `claude 2.1.195` (`write / idempotent / persist-across--resume / recolour-on-alias-switch`).

v1.9.2 — 2026-06-29 — Hermetic `CLAUDE_BIN` resolver test

Fixed

- `test_coverage.sh` B7 "**fallback when nowhere**" was **not hermetic** and failed on hosts that have a real `/usr/local/bin/claude` (caught live on `thinker.local`): the resolver *correctly* returns the system `claude` there, but the test wrongly assumed "`claude nowhere`" was achievable under a sandboxed `HOME/PATH` (it can't mask absolute system paths). Dropped that one assertion; the load-bearing discovery cases (explicit `CLAUDE_BIN`, `~/.npm-global/bin` discovery) stay covered. Runtime behavior unchanged.

Verified

- Suite **14/14 green on all five hosts** (this host, mistborn, thinker, amber, nezha); shellcheck 0.

v1.9.1 — 2026-06-29 — CLAUDE_BIN resolves across per-host install locations

A patch found during the live multi-host rollout of v1.9.0.

Fixed

- **Alias launches failed where claude was installed outside ~/.local/bin.** npm i -g @anthropic-ai/claude-code lands in different prefixes per host (~/.npm-global/bin, Homebrew, ~/.local/bin); the toolkit's fixed CLAUDE_BIN default mis-pointed on those hosts, so every claudeN/provider launch failed "No such file or directory" (amber.local needed a manual symlink to work). cma_resolve_claude_bin now prefers an explicit CLAUDE_BIN, then \$PATH, then the known locations (~/.local/bin, ~/.npm-global/bin, /opt/homebrew/bin, /usr/local/bin), with a ~/.local/bin fallback.

Verified

- Suite **14/14 green; shellcheck 0.** test_coverage.sh B7 covers explicit / npm-global / fallback resolution. v1.9.0's auto-session naming confirmed installed + **live-validated** (create-named + legacy-rename) on all five hosts: this host, mistborn, thinker, amber, nezha.

v1.9.0 — 2026-06-29 — Per-project auto-sessions that actually work live + zero-coverage tests

A minor release that makes the v1.8.0 "auto session-per-project" feature do what it promised. As shipped, opening any alias gave an **unnamed** session; three root causes — all reproduced and fixed against the real claude 2.1.195 binary, then proven **LIVE end-to-end** — are corrected here, plus test coverage for two zero-coverage utilities and a documentation refresh.

Fixed

- **Per-project auto-session naming never actually named the session — now proven LIVE.** Three independent root causes:
 - **Legacy/unnamed sessions were never renamed.** The launcher only `--resume`'d an existing session and never passed `--name`, so a session created by an older wrapper or by plain `claude` stayed unnamed forever. Fix: always pass `--name` on resume too. Proven live — `claude --resume <id> --name <x>` renames a previously-unnamed session (`custom-title <NONE> → <x>`), contradicting the docs but confirmed empirically.
 - **The session-existence check used a run-collapsing slug.** It collapsed runs of non-alnum to one `-` (`s/[^A-Za-z0-9]+/-/g`), but `claude` slugs **per char**, so paths with consecutive non-alnum segments (hidden dirs, `/tmp/.private`, `__pycache__`) false-negated the lookup and **re-created** instead of resuming. Fix: per-char slug (`s/[^A-Za-z0-9]/-/g`), matching the real on-disk dir names.
 - **`cma_run` self-heal regenerated only on a missing unset ANTHROPIC_ marker.** Wrappers predating auto-session carried that marker but not the `claude-session` one, so they never regained the integration. Fix: regenerate when **either** marker is missing.

Added

- **`docs/SESSION_COLOR.md`** — resolves the previously dangling reference, documenting per-project auto-session naming and the honest `/color` limitation: in `claude 2.1.195`, `/color` is **TUI-only** (no CLI flag, no `settings.json` key, no env var — verified against the binary and the docs), so the toolkit can only print a deterministic per-alias hint, never auto-apply it.
- **`test_toon.sh` (9 assertions) and `test_bootstrap.sh` (39 assertions)** — both utilities previously had **zero** coverage. `toon` (hermetic, SKIP-if-no-node): `toon.mjs` encode/decode round-trip, the `toon_encode.py` `python→mjs` chain, and non-zero exit on invalid JSON. `bootstrap` (hermetic): `claude-bootstrap --count 2 --yes` in a sandbox `$HOME` asserting account dirs, shared symlinks, private-file isolation, alias lines, and the documented refuse-to-clobber re-run behavior.
- **`test_coverage.sh` B6** asserts the emitted `cma_run / cma_run_provider` bodies actually carry the auto-session integration (bare-launch guard, `claude-session` flags, `eval set -- apply`, color hint) — the session script's own unit tests can't see the wrapper — plus a **self-heal regression**: a stale `cma_run` missing the `claude-session` marker is regenerated (exactly one `cma_run()`, provider-env isolation retained, aliases preserved). `test_session.sh` updated for `--name-on-resume` and a per-char-slug regression (a `/.cfg/` path must resume, not re-create).

- **Docs refresh.** README rewritten as a project landing page; scripts/README refreshed with the full current script inventory; the long-form guide gained "Per-project auto-session & per-alias color" and "TOON utility" sections; the /color notes pinned to the verified `claude 2.1.195` (superseding older `2.1.178` references).

Verified

- Full suite **14/14 ALL GREEN**; **shellcheck 0**. The session fix proven **LIVE end-to-end** on the real `claude 2.1.195` binary — fresh create, resume, and legacy-unnamed rename all confirmed. New tests are non-vacuous (concrete expected values / negative controls). Installed live on this host and validated.

v1.8.1 — 2026-06-29 — Merge-engine correctness + portability hardening

A patch release: an adversarial correctness audit of `claude-unify`'s merge engine plus a BSD/GNU portability pass over the test + proof tooling. All fixes/hardening — **no new features**.

Housekeeping: a divergent mirror lineage that re-created `v1.7.11` (`1e975e5`) was merged back into main resolved to **OURS** (local already carries `v1.7.11` → `v1.8.0` and later fixes that supersede it), leaving a tree byte-identical to HEAD so all four mirrors converge on one lineage; the containers submodule was fast-forwarded to latest main (`71d3256` → `67ed35a`).

Fixed

- **history.jsonl merge fused records across a source missing its trailing newline.** `merge_history_jsonl` cat'd sources into a temp first, gluing one file's last line onto the next file's first line → two entries collapsed into one invalid-JSON line. Fix: feed files straight to `awk` (fresh record per file). Regression **R1** (RED before, GREEN after).
- **enabledPlugins union dropped "any true".** The `jq` used `+/*` (rightmost-wins), so a plugin enabled in an earlier account but false in the lexically-last account ended up disabled for everyone — contradicting the documented "any true survives" guarantee. Fix: `OR-of-true` reduce over every account. Regression **R2**.
- **A single malformed settings.json aborted the whole unify — and naive guarding then risked silent config loss.** The multi-file `jq -s` ran unguarded under `set -e` (`settings` is item 15 of 16), halting mid-run. Merely skipping the merge was worse: `link_to_shared` still replaced each valid account's real `settings.json` with a symlink to a never-written target (a

dangling link → silent loss, exit 0). Final fix (hardened after adversarial review): validate each file with `jq empty` and merge only the valid ones (a malformed sibling is excluded, not fatal), and `link_to_shared` refuses to create a link when the shared target is absent. Regression **R3** (asserts the valid account's settings stay readable, not just that `unify` exits 0).

- **Directory-merge conflicts were resolved by lexical account name, not recency.** `merge_dir_into_shared`'s second pass overlaid only `ACCOUNTS[-1]` (alphabetically-last) while claiming to bias toward the "most recently active" account, so a stale account sorting last could clobber fresher memory/*.md. Fix: overlay every account with `rsync -au` so the newest-mtime file wins each conflict, independent of name/order. Regression **R4**.
- **Rollback left dangling symlinks.** Unify symlinks every shared item into each account; for an item an account never had there is no `.preunify` backup, so rollback's restore loop never visited it and the symlink dangled once `SHARED_DIR` moved aside. Fix: after restoring backups, remove any leftover symlink whose target points into `SHARED_DIR` (skipping the shared store itself). Regression **R5**.
- **Rollback restored a non-deterministic backup.** `find -print0` was unsorted, so when a path had several `.preunify.*` backups an arbitrary one was restored. Fix: `sort -z` (timestamps are `YYYYMMDDHHMMSS` = lexical-chronological) so the oldest — the true pre-unify original — wins. Regression **R6**.
- **test_unify.sh B2 was a vacuous PASS.** It called `cma_realpath` (a `lib.sh` function) without sourcing `lib.sh`, so the call errored to empty and the assertion compared `"" == ""` — the symlink target was never verified. Fix: `source lib.sh + set +e` (matching every sibling test that uses `lib` functions directly). Now prints the real resolved `SHARED_DIR/plugins/cache` path.
- **Portability: 3 GNU-only constructs broke the test/proof tooling on macOS** (the shipped runtime toolkit was already clean). `readlink -f` (no `-f` on BSD) in `assert_symlink_to/test_unify.sh` returned empty → spurious symlink pass/fail, fixed with a self-contained `_assert_realpath` in `assert.sh + cma_realpath` in `test_unify.sh`; `sed -E 's/\x1b.../'` (\xNN is GNU-sed-only) in `run-proof.sh/verify_opencode_live.sh` left ANSI in, skewing `grep -c` counts, fixed by building the ESC byte via `printf '\033'`; unguarded timeout (GNU coreutils) in `verify_opencode_live.sh`, fixed by resolving `timeout/gtimeout` once and degrading if absent.
- **De-vendored node_modules.** `node_modules/@toon-format/toon` was committed by an accidental "Auto-commit" yet load-bearing (`scripts/toon.mjs` imports the bare specifier; `toon_encode.py` shells out to it) with nothing ever running `npm install`. Removed from the tree (`git rm --cached + gitignore / node_modules/`). Proven by fresh-clone simulation: `ERR_MODULE_NOT_FOUND` before, encodes correctly after.
- **mktemp portability.** Standardized every bare `mktemp [-d]` to the templated `mktemp [-d] "${TMPDIR:-/tmp}/cma.XXXXXX"` form

CLAUDE.md prescribes (BSD mktemp requires a template; only GNU tolerates a bare call) across lib.sh, claude-unify/providers/opencode-sync/session/install, and the test harness (the sandbox keeps its cma-test. prefix for the cleanup safety check).

Added

- **B5 token-limit guard coverage** (test_coverage.sh). The v1.8.0 context_limit/max_output path (cma_provider_write_env → CMA_PROVIDER_CONTEXT_LIMIT/CMA_PROVIDER_MAX_OUTPUT → cma_run_provider exporting CLAUDE_CODE_MAX_OUTPUT_TOKENS) shipped with **zero** tests — the only v1.8.0 fix lacking one. 4 cases / 6 concrete-value assertions: round-trip (262144/32768), null→empty normalization, 7-arg back-compat, and the emitted wrapper carrying the export.
- **npm install step in install.sh** (soft — warns, never hard-fails, when npm is absent; core unify/add-account needs no Node), so a fresh clone gets @toon-format/toon without a vendored tree. curl-install.sh inherits it via delegation.
- **+16 regression assertions** — 6 in test_coverage.sh (B5) and 10 in test_unify.sh (R1–R6 above), each written RED-before / GREEN-after.
- **Documented two deliberate merge/sync trade-offs** in-code so they are explicit rather than silent: cma_merge_claude_json replaces (not element-unions) array values; claude-sync-state pull/push is last-writer-wins (no portable mutex is worth its stale-lock failure modes for a per-launch hook).

Verified

- Full suite **12/12 ALL GREEN; shellcheck 0**; all .py compile under python3 -W error. Each bugfix proven **RED-before / GREEN-after**; the de-vendor proven via fresh-clone simulation (node scripts/toon.mjs + toon_encode.py); ESC-strip verified functionally; the post-merge tree confirmed byte-identical to HEAD. Installed live on this host and validated against all existing aliases (3 native + 44 provider) + claude-list-accounts.

v1.8.0 — 2026-06-29 — Alias isolation + token-limit guard + per-project auto-sessions

A systematic-debugging pass fixing three reported issues plus a new session-per-project feature. Every root cause was reproduced and the fix proven with physical evidence before shipping.

Fixed

- **CRITICAL — aliases cross-contaminated API endpoints across sessions.** `cma_run_provider` exports `ANTHROPIC_BASE_URL/` `AUTH_TOKEN/MODEL/` `SMALL_FAST_MODEL` into the interactive shell, and native `cma_run` did **not** clear them — so running a provider alias (e.g. `xiaomi`) and then a native alias (`claude1`) in the same shell made the native one inherit the provider's endpoint (`api.xiaomimimo.com`). `cma_run` now unsets those four vars before launch. Proven live: after a leaked `xiaomi` env, native launch shows `ANTHROPIC_BASE_URL=<unset>`. Existing installs auto-regenerate the wrapper (migration keyed on the new unset `ANTHROPIC_` marker).
- **Token-limit 400 ("exceeded model token limit: 262144").** The `models.dev` catalog's per-model `limit.context /` `limit.output` were read for ranking but never emitted, so Claude Code overshot a provider's real context window. `providers_resolve.py` now emits `context_limit + max_output`; `providers_generate.py` and `cma_provider_write_env` write `CMA_PROVIDER_CONTEXT_LIMIT / CMA_PROVIDER_MAX_OUTPUT` into each `.env`; and `cma_run_provider` exports `CLAUDE_CODE_MAX_OUTPUT_TOKENS` from it. Proven: `kim-for-coding` now resolves `context_limit=262144` `max_output=32768` from the live catalog.
- **"workspace has not been trusted" warning on launch.** Confirmed NOT a merge bug (trust propagates across accounts correctly); the warned project was simply never trusted under any account. The launch wrapper now writes `projects[<root>].hasTrustDialogAccepted=true` for the launching project via the new `claude-session` helper.

Added

- **Auto session-per-project (`claude-session.sh`).** Every bare alias launch (native or provider) now resumes — or, the first time, creates — one long-lived Claude session per project root: stable `--session-id` (UUID derived from the `git-root` path), `--name` set to the root dir `basename` in lowercase `snake_case` (`Android 15` → `android_15`). Explicit args/flags are always respected verbatim. Verified against the real `claude` CLI: `--session-id` creates, `--resume` resumes.
- **Per-alias color hint.** A deterministic alias→color mapping over Claude Code's real palette (red blue green yellow purple orange pink cyan); printed as a `/color <x>` tip on launch. (Investigated thoroughly: `/color` is a TUI-only command with no CLI flag / settings key / writable persistence, so it cannot be auto-applied — the toolkit suggests it rather than faking it.)
- **`test_session.sh`** — 27 hermetic assertions for name/id/color/flags/trust/ `git-root` behavior. **`run-all.sh` is now 12 files / 60 assertions, ALL GREEN.**

Verified

- Full suite **12/12 ALL GREEN**; **shellcheck 0**; all .py compile under python3 -W error. All four items proven end-to-end against the live catalog and the emitted alias file.

v1.7.12 — 2026-06-28 — One-line curl installer

Added

- **curl-install.sh** — one-line bootstrap installer:

```
curl -fsSL https://raw.githubusercontent.com/vasic-digital/claude-toolkit/main/scripts/curl-install.sh | bash
```

Detects platform (Linux/macOS) and shell, auto-installs missing hard dependencies (jq, rsync, awk) via the system package manager (apt/dnf/apk/pacman/brew), clones (or pulls if already present) the repo with all submodules recursively to ~/claude-toolkit, runs install.sh, and prints next-steps. Idempotent; re-runnable. Install dir overridable via CLAUDE_TOOLKIT_DIR env var.

- **README.md** — curl one-liner added at the top of the Install section.
- **test_curl_install.sh** — 22 hermetic tests covering syntax, permissions, URL correctness, submodule cloning, idempotency, platform detection, dependency checks, error handling, and next-steps output.

Verified

- bash -n + shellcheck 0 on curl-install.sh and test_curl_install.sh.
- run-all.sh **11/11 ALL GREEN** (was 10; +test_curl_install.sh).

v1.7.11 — 2026-06-28 — Round-4: coverage-gap regression tests, toon recursion guard, arg validation

Fourth audit round: found the codebase is converging (export-docs, test harness, add/list/remove/rollback all verified clean); shipped 10 targeted coverage tests closing the same shallow-coverage class that let the v1.7.10 enable-plugins bug ship; plus 2 LOW code findings.

Fixed

- **toon_encode.py fallback_encode**: unbounded recursion on deeply nested JSON. A crafted input with >1000 levels would blow Python's default stack and exit with an unhandled traceback instead of encoding. Added a `_depth` guard; beyond 64 levels emits compact JSON as a safe fallback.
- **toon.mjs encode-file / decode-file**: running `toon.mjs encode-file` with no argument produced a confusing `TypeError` from Node's `fs.readFileSync`. Now prints `Error: encode-file requires a filename argument + exit 1`.

Added — coverage-gap regression tests (the same class that let enable-plugins

ship)

- **B9 (HIGH) — cma_ensure_alias_file migration path** (`test_coverage.sh`): builds a realistic old `cma_run_provider()` body lacking `claude-sync-state`, calls `cma_ensure_alias_file`, asserts the body is migrated, the following `alias claude1=` survives, and `cma_run_provider()` appears exactly once.
- **B3 (HIGH) — _cma_q bash quoting in cma_provider_write_env** (`test_coverage.sh`): sources a `.env` with a model name containing a literal single quote and asserts it round-trips intact; also asserts an injection payload does NOT execute on source (mirrors the already-tested Python `q()`).
- **B1 (HIGH) — absorb_default_plugins** (`test_unify.sh`): creates a real plugin file under `$HOME/.claude/plugins/cache/` before unify; asserts it lands in `$SHARED_DIR/plugins/cache/`.
- **B2 (HIGH) — link_default_plugin_subdirs** (`test_unify.sh`): asserts `$DEFAULT_DIR/plugins/cache` becomes a symlink into `$SHARED_DIR/plugins/cache` after unify, and that re-running unify doesn't create a second backup.
- **B4 (MEDIUM-HIGH) — sync_claude_md seed branches** (`test_unify.sh`): branch (b) seeds `$DEFAULT_DIR/CLAUDE.md` and asserts it wins; branch (c) removes it and gives an account a `CLAUDE.md`, asserts that one wins.

Verified

- `run-all.sh` **10/10 ALL GREEN** (coverage now $39+10=49$ assertions; unify now $43+7=50$); **shellcheck 0**; all `.py` compile under `python3 -W error`; `node --check toon.mjs` clean; `toon_encode 500-level-nest` no longer crashes; `toon.mjs missing-arg` gives clean error + exit 1.

v1.7.10 — 2026-06-28 — Round-3 audit: enable-plugins bug fix, path-traversal guards, proxy robustness

Third audit round (deep dive on the less-covered surface: `opcode_sync`, `claude-unify` merge, the `poe` proxy, `bootstrap`). Fixes verified centrally.

Fixed

- **`cma_enable_plugins` silently enabled NO plugins when given 3 or more** (`lib.sh`). The `jq --arg` index was derived as `${#args[@]}/2`, but each iteration appends **three** elements — so for the default 4 always-on plugins it produced arg names `p0,p1,p3,p4` while the `jq` program referenced `$p0..$p3`; `$p2` was undefined, `jq` failed, `2>/dev/null` swallowed the error, and `enabledPlugins` was left empty. Replaced the derived index with a dedicated counter. Proven live: `cma_enable_plugins a b c d` now yields all four `true` (was empty); a ≥ 3 -plugin regression test was added.

Security (defense-in-depth)

- **Path traversal via unvalidated provider id** in `claude-providers.sh` `cmd_show / cmd_remove`: `$id` was interpolated into `<dir>/$id.env` and then `cat/rm -f'd` without validation. Now rejected unless it matches `[A-Za-z0-9._-]` (blocks `../`), matching `cma_provider_write_alias`.
- **`opcode_sync.py` `${CLAUDE_PLUGIN_ROOT}` path traversal**: a malicious installed plugin could set an arg like `${CLAUDE_PLUGIN_ROOT}/../../../../tmp/evil.js` and `--enable-all-local` would have `OpenCode` exec the traversed path. Expansion now lexically contains the result to the plugin dir; an escaping value is left unexpanded (fails safe).
- **`cma_write_alias` now rejects whitespace in the config dir** (`lib.sh`): an unquoted space silently word-split the alias into a bogus command — now a clear error instead of a broken alias.

Robustness

- **`poe_proxy.py`**: `resolve_refs` gained a recursion-depth guard (a circular `$ref` previously crashed the request handler with `RecursionError`); the success-path `gzip.decompress` is now guarded like the error path, so a corrupt `gzip` body no longer propagates.

Verified

- `run-all.sh` **10/10 ALL GREEN** (coverage now 32 assertions incl. the enable-plugins + injection regressions); **shellcheck 0**; all `.py` compile under `python3 -W error`.
- `cma_enable_plugins` fix proven live (4 plugins → all true); `opcode` containment + id validation proven with PoCs. The model-verification / alias- write path is unchanged from v1.7.9's live-proven 137 models / 32 aliases.

Audit (round 3) — verified clean

`cma_merge_claude_json` private-key isolation, eval-token provenance, `cma_validate_alias`, proxy bind (localhost only) + no key logging, `_cma_q` escaping, `merge_settings_json` atomic write, history dedup, rollback NUL-safe traversal, bootstrap `--dir-of` injection filter. (`opcode_sync --enable-all` intentionally bypasses the needs-secret guard — operator opt-in, documented.)

v1.7.9 — 2026-06-28 — Hardening round 2: injection-safe alias writes, broadened secret redaction, docs accuracy, shellcheck 0

A second multi-agent audit + hardening pass on top of v1.7.8 (adversarial security audit + doc-accuracy audit + lint sweep, fixes verified centrally).

Security

- **Provider id / config dir can no longer inject shell via the alias file** (`lib.sh cma_provider_write_alias / cma_write_alias`). Both interpolate values into `alias name="..."` lines that the shell **re-parses on invocation**, and `jq @tsv` does not escape `"`. They now reject shell metacharacters (provider id restricted to `[A-Za-z0-9._-]`; config dir rejects `" $ \ \ ; & | < > ( )` and newline). Proven: `af00"; touch ...`` payload is rejected, no command runs, the hostile alias is never written.
- **Keys-file read no longer breakable by a quote in the path** (`claude-providers.sh cmd_sync_multi`). The old bash `-c "set -a; source '$keysf'; ..."` let a single quote in the keys-file path break out of the string. Replaced with an isolated subshell `( set +e; set -a +u; . "$keysf"; set +a; eval ... )` — the same safe pattern `cmd_sync` already used. Proven with a `do n't/` path.
- **.env value quoting** (`providers_generate.py`): `q()` now POSIX single-quote-escapes embedded quotes (mirrors `lib.sh _cma_q`), so a catalog value containing a quote can't inject when

cma_run_provider sources the .env. Proven: an injection payload is neutralized to a literal.

- **xtrace secret leak** (lib.sh cma_run_provider): the indirect key read is now wrapped in set +x/restore so an active set -x in the user's shell can't echo the key to the terminal or a redirected log.
- **Broadened secret redaction + guard**: cma_redact_secrets() (verify_opencode_live.sh) and the committed-proof scan guard (test_lib.sh) now also catch sk-ant-, hf_, AIza, xoxb-/xoxp-/xoxs-, pc-, re_, secret_, and JWTs — regardless of JSON field name — closing the gap where arbitrary MCP env-var names (e.g. NOTION_API_KEY) slipped through the original six-name allowlist.

Fixed

- **install.sh used readlink -f** (absent on BSD/macOS) for its symlink up-to-date check — missed by the v1.7.7 sweep. Now uses cma_realpath; the test_lib.sh guard scans install.sh too.
- **verify_aliases_live.sh hardcoded one developer's account dirs**, producing false FAILs on every other host. Now discovers accounts dynamically and skips dirs that don't exist.
- Dead code / cruft: providers_generate.py (unused import, dead vars, lambda→def, a no-op provider_id + ('' if ... else '')); model_verify.py (unused import hashlib); model_verify.py docstring --key → CMA_PROBE_KEY.

Docs

- Long-form doc + READMEs + CLAUDE.md corrected against the code: macOS rc-file caveat (~/.zshrc only), the test table now lists all 10 suites, the full installed-command list (+claude-providers/claude-sync-state/ claude-bootstrap), repo-relative paths (was ~/Documents/scripts/), a new claude-bootstrap section, the CMA_PROBE_KEY security model in §11, and a refreshed date stamp.

Quality

- **shellcheck: 93 → 0** across all scripts. Added .shellcheckrc (external-sources=true) which resolves the sourced-file warnings properly; fixed the \$?-after-condition (SC2319) test idioms, SC2015/SC1090/SC1003; the one remaining reserved no-op flag carries a justified inline disable.

Verified

- scripts/tests/run-all.sh **10/10 ALL GREEN**; **shellcheck 0**; every .py compiles under python3 -W error.

- Injection PoCs (provider id, config dir, .env value, keys-file path) all proven neutralized; broadened redaction proven against AIza/hf_/JWT/etc.
- Live sync --multi: **137 models verified, 32 aliases** across 8 providers (opencode 4, poe 33, chutes 7, huggingface 6, nvidia 30, openrouter 14, siliconflow 38, xiaomi 5), zero CMA_PROBE_KEY/unbound errors — identical to the v1.7.8 baseline, so the new key-read path is non-regressive end-to-end.
- 4-host byte-parity + 10/10 suite re-verified after deploy.

v1.7.8 — 2026-06-28 — Secret hygiene (argv + committed-proof leaks), dead-code fix, coverage tests

Security + robustness follow-up found by a parallel multi-agent audit of v1.7.7. Four independent subagents fixed disjoint file sets; integration + the full suite + live multi-model verification were run centrally.

Security

- **API key no longer passed on argv** (model_verify.py + claude-providers.sh). cmd_sync_multi invoked model_verify.py --key "\$token", placing the secret verbatim in /proc/<pid>/cmdline and ps aux output — readable by any user on a multi-user host. The key now flows via the CMA_PROBE_KEY environment variable (set per-command, not exported); model_verify.py reads it from the environment and errors clearly if unset. The --key flag is removed entirely.
- **API key no longer passed to curl on argv** (verify_aliases_live.sh). Six live-probe calls used -H "Authorization: Bearer \$key". The header is now written to a mktemp'd, chmod 600 config file consumed via curl --config (portable on GNU + BSD curl) and removed via an EXIT/INT/TERM trap.
- **Leaked secrets purged from committed proof artifacts** (committed in 24bc379, rolled into this release): the OpenCode live verifier wrote resolved opencode debug config /mcp list output — which contained a real provider key and a DB connection-string password — verbatim into the committed proof dir. The three artifacts are redacted; the generator (verify_opencode_live.sh) now redacts via cma_redact_secrets() before writing (raw dump → .raw temp → redacted file → .raw removed). **Operator follow-up still required:** rotate the leaked key and decide on a git-history scrub — the values remain in history on all four remotes.

Fixed

- **Unreachable code** in `verify_aliases_live.sh`: `exit $failed` sat *before* the Claude-alias test function and its caller, making them dead (shellcheck SC2317). `exit $failed` moved to the final statement.
- **Fragile \$? capture** in `test_list.sh`: `grep ...; [[ $? -ne 0 ]] then assert_eq 0 $? read $? from the wrong command. Now captures rc=$? immediately.`
- **Unquoted glob** in `claude-sync-state.sh`:67: `"$HOME"/${ACCOUNT_PREFIX}prov-*/` → `"$HOME/${ACCOUNT_PREFIX}"prov-*/` so only the intended `*` globs.
- **SyntaxWarning: invalid escape sequence '\ ' in** `providers_resolve.py`: the usage docstring's `\` line-continuations are now a raw string (`r"""`).

Added

- **test_coverage.sh** — 11 new hermetic tests (19 assertions) covering previously-untested `lib.sh` behavior: `cma_ensure_alias_file` (fresh / idempotent / old-format migration preserving unrelated lines), `cma_can_prompt` (`CMA_NONINTERACTIVE=1` and `no-tty` both non-interactive), `cma_enable_plugins` (JSON shape + additive + the `jq // falsy-vs-null` upgrade), `cma_link_shared_items` (every `CMA_SHARED_ITEMS` entry becomes a symlink into `$SHARED_DIR`, idempotent), and `stats-cache.json` newest-by-mtime selection.
- **Proof regression guard** (`test_lib.sh`): scans `scripts/tests/` proof for provider-key prefixes and URL `user:password@creds`, counting suspect lines so a failure never re-echoes a secret.

Verified

- `scripts/tests/run-all.sh` — **10/10 ALL GREEN** locally (was 9; +`test_coverage.sh`).
- Live multi-model verification (`claude-providers.sh sync --multi`, real HTTP probes with the host's real keys): **137 models verified, 32 aliases generated** across 8 providers (opencode 4, poe 33, chutes 7, huggingface 6, nvidia 30, openrouter 14, siliconflow 38, xiaomi 5). Zero `CMA_PROBE_KEY`-unset and zero unbound variable errors — the `env-var` key path works end-to-end. (Providers with 0 verified are external: dead/paid keys, HTTP 401/402/403, WAF blocks — not toolkit regressions.)
- The new proof secret-scan guard immediately earned its keep: on first cross-host run it flagged a **stale, pre-redaction proof dir on all three remote hosts** (3 files with literal secrets), which were then re-synced with the redacted artifacts.
- `model_verify.py` / `providers_resolve.py` compile clean under `python3 -W error`.

v1.7.7 — 2026-06-28 — Portable realpath (BSD portability hardening), set -u edge fix, regression tests

Follow-up hardening release found by a parallel multi-agent audit of v1.7.6.

Fixed

- **readlink -f** → **portable cma_realpath** at three sites: `claude-unify.sh` (`already_linked_to_shared` and `merge_settings_json`) and `claude-list-accounts.sh` (the link check). `readlink -f` is absent on older macOS and on other BSDs (FreeBSD/NetBSD); there the checks silently fail — making `claude-unify` re-link every shared item on each re-run (accumulating stale `.preunify.*` backups) and `claude-list-accounts` report linked accounts as "not linked". **Honest scope:** modern macOS (Sequoia) and GNU coreutils DO support `readlink -f`, so on the current fleet this was a *latent* bug with no active symptom — but it broke the toolkit's stated BSD portability. Replaced with a new pure-bash `cma_realpath` (single-arg `readlink symlink-walk + pwd -P`), verified to produce output identical to `readlink -f` on macOS.
- **set -u empty-array edge in cma_enable_plugins** — `jq "$ {args[@]}"` with an empty `args` is an "unbound variable" error on bash 3.2 (reachable via `CMA_ALWAYS_ON_PLUGINS=""` from the non-re-exec'd `claude-providers.sh`). Guarded with `${args[@]}+"$ {args[@]}"`.

Added

- `cma_realpath` portable canonicalizer in `lib.sh`.
- **Regression tests** (`test_lib.sh`): `cma_realpath` resolves a symlink chain and is identity on a real path; plus a guard asserting NO runtime script *invokes* `readlink -f`.

Verified

- `scripts/tests/run-all.sh` — **9/9 ALL GREEN on all four hosts:** nezha, thinker, amber (Linux), mistborn (macOS, re-exec to bash 5.3, BSD userland).
- `cma_realpath` output confirmed byte-identical to `readlink -f` on macOS.

Audit findings (v1.7.6 — no code change required)

- Disabled providers are EXTERNAL, not toolkit bugs (toolkit correctly disabled them on failed verify): `github-models` → HTTP

401 (dead GitHub PAT), upstage → HTTP 403 from AWS WAF (egress-IP block).

- `api_keys.sh` across all 4 hosts: **0 dangling refs, 0 duplicates, 0 malformed**; key parity confirmed (mistborn's 2 host-local Kimi-Platform keys preserved).
- Cross-host integrity: all 11 toolkit scripts byte-identical to the released tag on every host.
- Known/deferred: published tags v1.2.0 (gitlab) and v1.5.0 (gitlab/gitverse/gitflic) point to older commits than local — reconciling needs a force tag push; left for a maintainer decision.

v1.7.6 — 2026-06-28 — Always-non-interactive execution, alias-file integrity, macOS/bash-3.2 portability, 4-host rollout

Fixed

- **Alias-file corruption from a mis-firing migration** — `cma_ensure_alias_file`'s "outdated `cma_run_provider`" migration grepped for `claude-sync-state` pull, but the emitted on-disk text is `.../claude-sync-state" pull` (a quote precedes the space), so the guard **never matched** and the migration fired on *every* alias write. Its `awk` then chopped everything from `cma_run_provider()` to EOF — destroying previously-written provider aliases and any `claudeN` aliases that follow the function block. This silently corrupted the alias file on multi-provider / multi-account hosts. Detection is now scoped to the function body and matches the bare command name (quote/space agnostic), and the migration removes **only** the function block, preserving alias lines. This was the single root cause of the failures across `test_providers.sh`, `test_claude.sh`, and `test_add_remove.sh`.
- **set -u abort while sourcing the keys file** — provider sync sourced `~/api_keys.sh` inside a `set -euo pipefail` subshell. A dangling reference in the user's keys file (e.g. `export SARVAM_API_KEY=$ApiKey_Sarvam_AI_India`) aborted the source **mid-file** under `nounset`, leaving every key defined *after* it unexported — so those providers silently failed verification ("unverified") and `stderr` was spammed with "unbound variable". Keys are now sourced with `nounset` disabled (subshell-local in sync; save/restore around the alias-file `cma_run_provider`). Installed alias files are auto-migrated to the `nounset-safe` wrapper on next sync.
- **macOS / bash-3.2 portability of the test harness** — `tests/run-all.sh` used `mapfile` (bash 4+), so the **entire suite failed to run on stock macOS**. Replaced with a portable read loop

and guarded empty-array expansion under `set -u`. Same fix applied to `test_lib.sh` and `tests/lib/sandbox.sh` (empty `${arr[@]}` expansions are unbound on bash 3.2). The suite now runs green on macOS bash 3.2.

Added

- **CMA_NONINTERACTIVE + automatic TTY detection** — a new `cma_can_prompt` helper makes every prompt (`claude-add-account`, `claude-remove-account`, `claude-bootstrap`) fall back to its non-interactive default whenever no terminal is available (CI, SSH without a PTY, the test sandbox) or when `CMA_NONINTERACTIVE=1` is exported. Toolkit execution is now **always non-interactive off a terminal**. Destructive account removal still refuses (rather than guessing) without `--yes` when it cannot confirm.
- **Regression tests** for non-interactive `claude-add-account` and for alias-line survival across repeated account adds.
- **test_export.sh graceful SKIP** when its prerequisites (`pandoc` + a PDF engine) are absent — matching the existing SKIP convention for optional-dependency features.

Multi-host rollout (nezha · mistborn.local · thinker.local · amber.local)

- Distributed `~/api_keys.sh` to every host via a **no-loss merge** (host-local keys preserved — e.g. `mistborn` kept its 2 Kimi-Platform keys; `amber` created fresh) and wired **both** `.bashrc` and `.zshrc` to source it on every host.
- Installed/updated the toolkit on all four hosts and configured `claude1/claude2/claude3` on each; installed Claude Code on `amber`.
- Ran live provider/model detection on every host — **17-20 active providers each**, models verified via HTTP probes, **0 unbound errors**.

Verified

- `scripts/tests/run-all.sh` — **9/9 files, ALL GREEN on all four hosts**: `nezha` (Linux), `thinker` (Linux), `amber` (Linux), `mistborn` (macOS / bash 3.2).
- Cross-host: both rc files source `api_keys.sh`; `claude1/2/3` + `poe/deepseek/xiaomi` aliases present on every host.

v1.7.5 — 2026-06-28 — Cross-provider / resume session visibility fix

Fixed

- **Cross-provider /resume session loss** — when switching between provider aliases (e.g., deepseek → opencode → kimi-for-coding), /resume would sometimes show empty session history. Root cause: the `cma_run_provider` function in the alias file was **missing sync-state pull/push calls** that were present in `lib.sh`. The alias file is what actually runs when a user invokes an alias, so the sync never happened.
- **Migration for outdated alias files** — added automatic detection and regeneration of outdated `cma_run_provider` functions in `lib.sh`. If the function exists but lacks `claude-sync-state pull`, it's removed and rewritten with the correct implementation.
- **Router transport transformer config** — added `transformer: {use:["cleancache","streamoptions"]}` to the alias file's router transport section (was only in `lib.sh`), ensuring `cache_control` stripping works for all router-transport providers.

Root Cause Analysis

The `cma_run_provider` function in `lib.sh` (lines 225-333) correctly includes sync-state pull/push calls, but the alias file's copy of the function was outdated and explicitly stated "cross-account claude-sync-state is intentionally NOT run." This meant:

1. Sessions created under provider A had their `lastSessionId` written only to A's `.claude.json`
2. When switching to provider B, B's `.claude.json` still had its own (different) `lastSessionId`
3. `/resume` read B's `lastSessionId` and couldn't find A's session

After fix: all providers/accounts share the same merged `lastSessionId` via sync-state.

Verified

- **Local host:** all providers show identical `lastSessionId` after sync (confirmed)
- **mistborn.local:** 76 projects merged across all accounts/providers (confirmed)
- **Migration:** `install.sh` correctly detects and fixes outdated alias files on both hosts

Tests

- Cross-alias session visibility (Section 5): **ALL PASS**
- Existing test suite: session-related tests pass

v1.7.4 — 2026-06-26 — Kimi provider fix + AWS IaC MCP disabled by default

Fixed

- **Kimi Code provider base URL** in scripts/providers/overrides.json — changed from /coding/v1 to /coding/ so native transport works correctly.
- **AWS IaC MCP timeout** — removed aws-dev-toolkit/awsiac from the default OpenCode MCP allowlist in scripts/claude-opencode-sync.sh. The server consistently timed out on connection and is now configured but disabled by default.

Changed

- Regenerated Claude_Multi_Account_Fine_Tuning.{html,pdf,docx} from current markdown.
- Refreshed proof artifacts in scripts/tests/proof/.

Tests

- Local: **9/9 ALL GREEN**
- Live OpenCode verification: **9 passed, 0 failed**, 27/27 enabled MCPs connected
- Provider alias verification: **5 passed, 0 failed**

v1.6.6 — 2026-06-21 — TOON integration for token-efficient prompts

Added

- **TOON (Token-Oriented Object Notation)** integration — saves ~40% tokens vs JSON for structured data in LLM prompts by declaring fields once in arrays.
- **scripts/toon.mjs** — Node.js TOON utility (encode/decode/demo)
- **scripts/toon_encode.py** — Python wrapper for TOON encoding
- **docs/TOON_Integration.md** — comprehensive guide on using TOON with Claude Code
- **package.json** — @toon-format/toon v2.3.0 dependency

Token Savings

- File listings: ~39% fewer tokens
- Tool definitions: ~40% fewer tokens
- User records: ~42% fewer tokens
- Accuracy: 76.4% (vs JSON's 75.0%)

Note

TOON formats message CONTENT for token savings. API transport remains JSON (providers require it). HTTP/3 and compression require provider-side support.

Tests

- 8/8 ALL GREEN

v1.6.5 — 2026-06-21 — Poe proxy fix (alias file + install)

Fixed

- **Poe proxy not starting from alias** — proxy logic was only in `lib.sh`, not in the alias file's `cma_run_provider` function. The alias file is what actually runs when a user invokes an alias. Added proxy detection + auto-start to the alias file.
- **install.sh: SHARE_DIR → SHARED_DIR** — wrong variable name caused unbound variable error on nezha (Linux, set -u).
- **install.sh: auto-copy proxy scripts** to `~/.local/share/.../proxy/` during install.

Verified

- All 3 Poe aliases work: poe , poe2 , poe3 
- Deployed to both local host and nezha.local

Tests

- 8/8 ALL GREEN

v1.6.4 — 2026-06-21 — Poe proxy fix for tool compatibility

Fixed

- **Poe tool format error** — Poe requires parameters in every tool function definition. Claude Code sometimes omits it (valid in Anthropic format, invalid for Poe). Added `poe_proxy.py` that auto-fixes tools before forwarding to Poe API.
- **Proxy auto-start** — `cma_run_provider` now auto-starts compatibility proxies for providers that need them (detected by `scripts/proxy/<provider>_proxy.py`).

Verified

- All 3 Poe aliases work through proxy: poe , poe2 , poe3 

Tests

- 8/8 ALL GREEN

v1.6.3 — 2026-06-21 — Poe provider (382 models, 3 aliases)

Added

- **Poe provider** — universal AI platform with 382 models from all major providers. OpenAI-compatible API at <https://api.poe.com/v1>. Chat, code, image gen, video gen, TTS, STT, and more.
- **3 aliases**: poe (claude-sonnet-4.6 + gpt-5.4-mini), poe2 (gpt-5.5 + deepseek-v4-pro-e), poe3 (grok-4 + gemini-3.1-pro)
- **key-aliases**: POE_API_KEY + ApiKey_Poe → poe
- **Tool calling verified** on claude-sonnet-4.6, gpt-5.4-mini, deepseek-v4-pro-e, grok-4
- **382 models categorized**: 130 chat/reasoning, 16 code, 40 image gen, 17 video gen, 12 TTS, 1 STT, 166 other
- **Documentation**: full Poe section in `Provider_Aliases_User_Guide.md`

Verified

- API endpoint responds correctly
- Authentication works
- Tool calling confirmed

- All 3 aliases tested through ccr with "Do you see our codebase?" — all YES

v1.6.2 — 2026-06-21 — Chutes provider documentation + model update

Changed

- **Chutes provider models updated** — catalog was stale. Chutes now offers 13 TEE (Trusted Execution Environment) models. Updated strong=zai-org/GLM-5.2-TEE, fast=Qwen/Qwen3.6-27B-TEE.
- **Chutes documentation** added to Provider_Aliases_User_Guide.md with full model table, TEE explanation, pay-per-use note, and setup instructions.

Verified

- Chutes API endpoint responds correctly
- All 13 TEE models accessible (require funded account for actual inference)
- OpenAI-compatible format confirmed at <https://llm.chutes.ai/v1>

v1.6.1 — 2026-06-21 — cache_control fix + E2E tests

Fixed

- **cache_control parameter error** — Claude Code sends cache_control (Anthropic-specific) in its API requests. ccr forwarded this to OpenAI-compatible endpoints which reject it with HTTP 422. Fixed by adding ccr's built-in cleancache transformer to every provider config, which strips cache_control before forwarding to the provider.

Added

- **alias_e2e_test.py** — end-to-end alias verification script that tests each alias by sending requests through ccr and verifying responses work without errors.

Verified working (all aliases tested with "Do you see our codebase?")

- `opencode` (north-mini-code-free):  YES
- `opencode2` (big-pickle):  YES
- `opencode3` (nemotron-3-ultra-free):  YES
- `deepseek` (native transport):  YES
- `deepseek2` (router transport):  YES
- `xiaomi` (native transport):  YES
- `zai-coding-plan` (router transport):  YES

v1.6.0 — 2026-06-21 — Multi-alias provider system

Added

- **Multi-alias provider system** — every provider can now have multiple aliases (`provider`, `provider2`, `provider3`...) exposing ALL working models, not just the top 2. Verified via live HTTP probes with anti-bluff detection.
- **`model_verify.py`** — comprehensive model verification & scoring engine. Tests every model for a provider via HTTP probes, scores on 7 dimensions (existence 25pts, `tool_call` 20pts, reasoning 15pts, `context_window` 15pts, streaming 10pts, latency 10pts, `free_tier` 5pts). Anti-bluff detection prevents false positives (HTTP 200 with error body, empty responses, boilerplate errors). 24h verification cache to avoid re-testing.
- **`providers_generate.py`** — multi-alias generation from verified models. Pairs models into alias groups of 2 (strong + fast), handles odd count (last model reused for both positions), single model (used for both positions). Generates env files, shell aliases, and `overrides.json` entries.
- **`claude-providers.sh --multi`** — new flag for sync that triggers the full verification + multi-alias generation pipeline. Additional flags: `--max-aliases` (default 5), `--min-score` (default 25), `--verify-concurrency` (default 5).
- **Endpoint normalization** — `/anthropic` endpoints auto-converted to `/v1` for OpenAI-compatible probing during verification.
- **Submodules updated** to `helix_translate-2.3.1`: `LLMsVerifier` (ModelVerifier, Seed, xiaomi provider), challenges (anti-bluff §11.4, chaos/stress tests), containers (deploy-stack).

Changed

- Probe `max_tokens` increased from 32 to 128 — reasoning models need more tokens for chain-of-thought + response (was causing false anti-bluff rejections).

- User-Agent header added to HTTP probes (some APIs require it).

Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures.

Usage

```
# Standard sync (2 models per provider, as before)
claude-providers sync

# Multi-alias sync (verify ALL models, create multiple aliases)
claude-providers sync --multi

# With options
claude-providers sync --multi --max-aliases 10 --min-score 20
```

v1.5.1 — 2026-06-20 — Linux stat fix + nezha deployment

Fixed

- **stat -f %m on Linux** — the mtime cache check in `claude-providers.sh` used `stat -f %m || stat -c %Y` as an `||` chain. On Linux, `stat -f` succeeds (returning filesystem info, not mtime), so both outputs merged into garbage ("File: ...1781634386"), causing File: unbound variable under set -u. Fixed with case "\$ (uname -s)" to pick the correct flag per platform.

Deployment

- **nezha.local** (Linux x86_64) deployed and verified: 19 providers activated, 100/100 provider tests pass, 5/5 live verifier pass, cross-alias sync confirmed. Evidence in `scripts/tests/proof/90-nezha-deployment.txt`.

Full test suite

- macOS: 8/8 ALL GREEN
- Linux (nezha): 7/8 pass (export fails: pandoc not installed — pre-existing)

v1.5.0 — 2026-06-20 — Cross-alias session visibility

Added

- **Cross-alias session visibility** — sessions created under ANY alias (claudeN, deepseek, opencode, xiaomi, etc.) are now visible from every other alias via /resume. Memory, project settings, and session data are fully shared across all accounts and providers.
- **claude-sync-state.sh extended** — now discovers provider dirs (~/.claude-prov-*) alongside account dirs for its .claude.json merge. Provider sessions participate in the same lightweight jq merge that keeps account sessions in sync.
- **cma_run_provider sync-state hooks** — the provider wrapper now calls claude-sync-state pull before launch and claude-sync-state push after exit, matching the cma_run pattern. Previously provider sessions were intentionally excluded from sync; now they participate fully.
- **Sandbox test coverage:** 10 new assertions proving cross-alias merge (sessions from account→provider, provider→account, account→account all visible after sync). Providers test 90 → 100 assertions.
- **Live verification:** lastSessionId for a real project confirmed identical across all dirs (3 accounts + 1 provider). 61 projects merged in every .claude.json. Evidence in scripts/tests/proof/80-cross-alias-sessions.txt.

Changed

- scripts/claude-sync-state.sh — provider dirs included in merge targets
- scripts/lib.sh — cma_run_provider wrapper updated with sync-state pull/push
- Alias file aliases.sh — updated cma_run_provider function (re-installed)

Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures. Providers test includes 10 new assertions for cross-alias session visibility.

How it works

1. claude-sync-state pull merges every account's + provider's .claude.json into the launching dir before Claude Code starts (including lastSessionId, allowedTools, MCP config, etc.).

2. Claude Code launches with the merged state — `/resume` sees all sessions.
3. `claude-sync-state` push merges the post-session `.claude.json` back out after exit, so the next alias to launch picks up the new session.
4. The `sessions/` directory was already shared via symlink — this release ensures `.claude.json` project settings are also merged.

Performance

- Adds ~1-2 seconds overhead per provider launch (jq merge of `.claude.json` across all dirs). Same overhead that `claudeN` aliases already have.

v1.4.0 — 2026-06-20 — OpenCode Zen provider alias

Added

- **opencode provider alias** — OpenCode Zen curated AI gateway with **21 free models** (all \$0 cost, all support tool calling + reasoning) and 49 paid models. The alias uses **router transport** (ccr) targeting the OpenAI-compatible endpoint `https://opencode.ai/zen/v1/chat/completions`.
- **Model overrides**: `strong` = `big-pickle` (free stealth model, 200K context, reasoning + tool_call), `fast` = `deepseek-v4-flash-free` (free, 200K context, reasoning + tool_call). Pinning is deliberate — auto-selection would pick `nemotron-3-ultra-free` (1M ctx) as `strong` and `trinity-large-preview-free` (131K, no reasoning) as `fast`, both suboptimal for coding workloads.
- **key-aliases.json mappings**: `ZEN_API_KEY` → `opencode` and `ApiKey_Opencode_Zen` → `opencode` (both key vars present in the user's keys file).
- **overrides.json pin**: `strong_model=big-pickle`, `fast_model=deepseek-v4-flash-free` (no `transport/base_url` override needed — catalog values are correct).
- **Sandbox test coverage**: resolver tests (key-alias mapping for both key vars, router transport from `@ai-sdk/openai-compatible` npm, `zen/v1` `base_url` from catalog, model override beats auto-selection, stale-model-never-selected guards) + sync e2e tests (env file, alias, config-dir + plugins symlink, account-detection exclusion, idempotency, no-secret-leak). Providers test 69 → 90 assertions.
- **Live endpoint verification**: GET `/v1/models` HTTP 200; POST `/v1/chat/completions` round trip HTTP 200 with correct text for `big-pickle` (stealth, cost=\$0, reasoning_content present) and `deepseek-v4-flash-free` (cost=\$0); additional free models (`mimo-v2.5-free`, `nemotron-3-ultra-free`, `north-mini-code-free`) all HTTP

200 with cost=\$0. Evidence in scripts/tests/proof/70-zen-live.txt (secret-free).

- **Docs:** dedicated opencode section in docs/Provider_Aliases_User_Guide.md (full free models table, setup, usage, live-verified notes, stealth model explanation).

Changed

- scripts/providers/key-aliases.json and scripts/providers/overrides.json extended with the opencode entries (config-only; no code changes — same dynamic pattern as Xiaomi v1.3.0 / Z.AI v1.2.0 / DeepSeek).

Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures. Providers test includes 21 new assertions for opencode.

Honest notes

- The alias uses router transport (ccr) because Zen's free models use OpenAI-compatible format (/v1/chat/completions), not Anthropic native format. This adds a ccr dependency that native-transport aliases (deepseek, xiaomi) don't have.
- Big Pickle is a stealth model — the actual model served may vary (observed as deepseek-v4-flash). This is by design per OpenCode's documentation.
- The same pre-existing ~/api_keys.sh set -u issue affects the in-process verifier for all providers; authoritative proof is the direct HTTP round trip.
- The 2 pre-existing, environmental opencode-skill-discovery failures in run-proof.sh remain unchanged (unrelated to this work).

v1.3.0 — 2026-06-19 — Xiaomi MiMo provider alias

Added

- **xiaomi provider alias** — Xiaomi MiMo via the **Anthropic-native endpoint** <https://api.xiaomimimo.com/anthropic> (POST /anthropic/v1/messages). Unlike most providers in this toolkit, MiMo exposes a genuine native Anthropic endpoint that accepts Authorization: Bearer, so the alias uses **native transport** with no claude-code-router (ccr) dependency — the same direct-launch model as deepseek.

- **Model overrides:** strong = mimo-v2.5-pro (flagship, 1M context, reasoning, tool-call), fast = mimo-v2-flash (256K, cheapest tier). Pinning is deliberate — models.dev lists a mimo-v2.5-pro-ultraspeed id the **live API does not serve**, so the override guarantees only live-served ids are used.
- **key-aliases.json mapping:** XIAOMI_MIMO_API_KEY → xiaomi (the user's key-var name does not match the models.dev provider's documented XIAOMI_API_KEY env).
- **overrides.json pin:** native transport, /anthropic base_url, mimo-v2.5-pro / mimo-v2-flash.
- **Sandbox test coverage:** resolver tests (key-alias mapping, override forces native transport, /anthropic base_url beats catalog /v1, model pinning beats the stale ultraspeed entry, stale-id-never-selected guard) + sync e2e tests (env file, alias, config-dir + plugins symlink, account-detection exclusion, idempotency, no-secret-leak). Providers test 60 → 69 assertions.
- **Live endpoint verification:** GET /v1/models HTTP 200 (10 models); native /anthropic/v1/messages round trip HTTP 200 with correct text for both mimo-v2.5-pro and mimo-v2-flash; tool calling proven (finish_reason: tool_calls ◦ reasoning_content); streaming confirmed. Evidence in scripts/tests/proof/60-xiaomi-live.txt (secret-free).
- **Docs:** dedicated xiaomi section in docs/ Provider_Aliases_User_Guide.md (model table, setup, usage, live-verified notes).

Changed

- scripts/providers/key-aliases.json and scripts/providers/overrides.json extended with the xiaomi entries (config-only; no code changes — same dynamic pattern as Z.AI v1.2.0 / DeepSeek).

Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures. Providers test includes 9 new assertions for xiaomi. Live provider verifier 5/5 PASS.

Honest notes

- The only failures in the repo's run-proof.sh are 2 **pre-existing, environmental** opencode-skill-discovery checks, unrelated to Xiaomi (zero opencode files changed by this release; they fail identically when run standalone).
- The in-process LLMsVerifier step reports (unverified) for every provider because ~/api_keys.sh has a pre-existing unrelated unbound variable on a different provider's key under set -u;

authoritative proof is the direct native-endpoint round trip (HTTP 200), recorded in the evidence file.

v1.2.0 — 2026-06-19 — Z.AI Coding Plan provider alias

Added

- **zai-coding-plan provider alias** — OpenAI-compatible router transport via `https://api.z.ai/api/coding/paas/v4` (Coding Max-Yearly Plan endpoint).
- **Model overrides**: strong = glm-5.2 (flagship 1M context reasoning model, free on plan), fast = glm-4.7 (204k context, tool_call, 0 cost).
- **key-aliases.json mapping**: ZAI_API_KEY → zai-coding-plan (targets the coding plan API endpoint instead of the general z.ai paas endpoint).
- **overrides.json pin**: overrides auto-selected strong/fast models for the coding plan.
- **Sandbox test coverage**: resolver tests (env-key matching, coding endpoint, router transport, glm-5.2/glm-4.7 model selection) + sync e2e tests (env file, alias, model overrides).
- **Live endpoint verification**: HTTP 200 at /models (8 models discovered), curl test of glm-4.7 chat completion confirmed operational.
- **ccr integration**: provider auto-registered in `~/.claude-code-router/config.json` as the active default route.

Changed

- `overrides.json` extended with zai-coding-plan section for model pinning.

Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures. Provider tests include 5 new assertions for zai-coding-plan.

v1.1.0 — 2026-06-16 — Distributed infrastructure + provider verification

Headline: stand up the full LLMsVerifier System on a remote host for heavy testing against **real production LLM services**, plus end-to-end provider aliases proven on two hosts and two transports.

Added

- **containers + challenges submodules** (submodules/) — the distributed-boot orchestrator and its sibling. `helix-deps.yaml` confirms containers has zero own-org submodule deps.
- **Remote host registration** — `config/containers/nezha.env` registers `nezha.local` as a remote boot/test host (SSH key, podman runtime).
- **LLMsVerifier deployment overlays** (`config/containers/llmsverifier/`):
 - `docker-compose.app.yml` — the `llm-verifier` API (cgo image, config mount, `/api/health` healthcheck, loopback, fail-fast secrets).
 - `docker-compose.infra.yml` — observability tier: prometheus + grafana (auto-provisioned datasource + dashboard) + node-exporter. **No DBs** (the app uses SQLite; postgres/redis were unused and removed).
 - `Dockerfile.nezha` / `Dockerfile.mv` — cgo nested-module builds for the server + the model-verification tool.
 - `patches/0001..0005` — upstream LLMsVerifier fixes (see PR #2 below).
- **Deployment guide** `config/containers/llmsverifier/README.md` and the **Provider Aliases User Guide** `docs/Provider_Aliases_User_Guide.md` (HTML/PDF/DOCX exports included).
- **QA evidence** `docs/qa/20260616-infra/` — verification proofs, endpoint coverage, security posture, observability, per-provider sweeps, dual-host end-to-end alias proofs.

Changed

- **Provider session accent color: orange → purple** across spec, guide, and the long-form doc. (Claude Code 2.1.178 cannot persist a default `/color`, so this is the documented default + a manual `/color purple` — a platform limit.)
- `claude-add-account` consolidated onto the shared `cma_link_shared_items` helper (single `CMA_SHARED_ITEMS` source).
- `claude-export-docs` now also emits **DOCX** (HTML/PDF/DOCX).

Fixed (LLMsVerifier — shipped as PR #2, applied to deployed builds)

- **Auth header missing** — verification requests sent no Authorization header → HTTP 401 for every provider. Now Bearer <key>.
- **cohere 405** — switched to the OpenAI-compatible endpoint (`api.cohere.ai/compatibility/v1`). Verifies at score 1.00.
- **gemini / huggingface** — corrected to OpenAI-compatible / router endpoints (huggingface verifies; gemini code-ready pending a valid key).

- **model-id strictness** — verifies a requested id directly when not in the discovered list (no premature `model_not_found`).
- **no /metrics** — added `GET /api/metrics + /metrics` (stdlib Prometheus).
- **provider-session sync-state noise** — `cma_run_provider` no longer runs cross-account sync-state on isolated provider dirs.

Verified live (real "Do you see my code?" against production APIs)

- **9 providers verified:** DeepSeek, Groq, Mistral, Cerebras, Novita, NVIDIA, Cohere, Codestral, HuggingFace.
- **Both transports, both hosts:** native (DeepSeek) + router (Novita via ccr) on macOS and on nezha.
- Account-side failures (402/401/429/403) and non-OpenAI providers documented honestly; excluded under "valid users only" but kept fully supported.

Safety

- Provider dirs (`~/.claude-prov-*`) excluded from account detection — existing claudeN accounts and `claude-add-account` untouched.
- Secrets only in the keys file + on-host `mode-600 .env`; never in the repo. All published ports bound to loopback.

v1.0.0 — 2026-06-16 — Dynamic provider-alias generator

First tagged release. `claude-providers` creates per-provider Claude Code aliases (DeepSeek, Groq, GLM, ...) from your keys file pointed at each provider's strongest model — fully dynamic via `models.dev` + the `LLMsVerifier` submodule, hybrid native/claude-code-router transport, full lifecycle + tests + docs. See `docs/Provider_Aliases_User_Guide.md`.

v1.6.7 — 2026-06-21 — Poe proxy fix for all aliases

Fixed

- **Poe proxy not starting for poe2/poe3** — proxy detection used exact provider ID (`poe2_proxy.py`) which doesn't exist. Fixed to check base name too (`poe_proxy.py` for poe2, poe3 aliases).
- **lib.sh:** base proxy detection with `${CMA_PROVIDER_ID%[0-9]*}`

- **alias file:** same fix applied

Verified

- All 3 Poe aliases work: poe , poe2 , poe3 
- Deployed to both local host and nezha.local

Tests

- Local: 8/8 ALL GREEN
- nezha.local: 7/8 (pandoc missing — pre-existing)

v1.6.8 — 2026-06-21 — Poe proxy gzip fix

Fixed

- **Poe proxy gzip decompression** — Poe API returns gzip-compressed responses but the proxy tried to read them as UTF-8 without decompressing, causing UnicodeDecodeError. Added gzip decompression for both success and error responses.

Verified

- poe (claude-sonnet-4.6):  YES
- poe2 (gpt-5.5):  YES
- poe3 (grok-4):  Different error (Grok-4 schema validation, not tools format)

Tests

- Local: 8/8 ALL GREEN
- nezha.local: 8/8 ALL GREEN

v1.6.9 — 2026-06-21 — Poe proxy \$ref fix for Grok-4

Fixed

- **Poe proxy \$ref resolution** — Claude Code sends tool schemas with \$ref references to \$defs. Grok-4 and some providers don't support \$ref in tool schemas. Added

`resolve_refs()` function that extracts `$defs`, resolves all `$ref` references to inline definitions, and removes `$defs`.

Verified

- `poe (claude-sonnet-4.6)`:  YES
- `poe2 (gpt-5.5)`:  YES
- `poe3 (grok-4)`:  YES (was failing, now works)

Tests

- Local: 8/8 ALL GREEN
- `nezha.local`: 8/8 ALL GREEN

v1.7.0 — 2026-06-22 — Poe proxy complete fix (all aliases verified)

Fixed

- **Poe proxy shared directory** — the proxy at `~/.local/share/.../proxy/poe_proxy.py` was the OLD version without `gzip` and `$ref` fixes. `install.sh` copies from `scripts/` but the shared dir still had the old version. Fixed by ensuring updated proxy is copied to shared directory.
- **install.sh** now copies proxy scripts during installation (already in place)

Verified (all three aliases through full Claude Code flow)

- `poe (claude-sonnet-4.6)`:  YES
- `poe2 (gpt-5.5)`:  YES
- `poe3 (grok-4)`:  YES

Root Cause Analysis

The proxy had three issues:

1. **gzip** — Poe returns gzip-compressed responses, proxy didn't decompress
2. **\$ref** — Claude Code sends tool schemas with `$ref`, Grok-4 doesn't support them
3. **shared dir** — Updated proxy wasn't copied to shared directory

All three fixed and verified.

Tests

- Local: 8/8 ALL GREEN
- nezha.local: 8/8 ALL GREEN

v1.7.1 — 2026-06-22 — Full validation + release

Fixed

- **Port-ready check** for proxy startup — replaced sleep 1 with polling loop (`lsof -i`) ensuring proxy is listening before ccr config is written
- **Claude alias regression test** — 11 assertions proving claudeN aliases use `cma_run` (no proxy/transformer code), providers use `cma_run_provider`
- **Command injection fix** in `verify_aliases_live.sh` — replaced `bash -c` subshell with safe indirect expansion

Tests

- Local: **9/9 ALL GREEN** (new: `test_claude.sh` — 11 assertions)
- nezha.local: 8/9 (export fails — pandoc missing)

Release

- v1.7.1 — pushed to github, gitlab, gitflic, gitverse

v1.7.2 — 2026-06-22 — Claude alias verification, full release

Added

- **Claude alias verification** in `verify_aliases_live.sh` — tests claude1/2/3 alongside provider aliases
- **TOON tested** on all aliases — verified working

Tests

- Local: **9/9 ALL GREEN**
- nezha.local: 8/9 (pandoc missing)
- All claude1/2/3:  OK
- All provider aliases: verified